

Predictive Modeling: Classification

CIS 635 — Knowledge Discovery & Data Mining
Concepts, Core Algorithms, and Evaluation

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [8] this material when using or reproducing it.

Unit: Predictive Modeling — Classification

This is an introductory reading on classification. It is the companion to **Predictive Modeling: Regression** (Reading 5.1), whose modeling workflow, train/validation/test protocol, leakage rules, and bias–variance discussion it reuses rather than repeats. It also draws on the EDA readings, especially distance measures and the curse of dimensionality (Part 2, Sections 3 and 3.4), the base-rate intuition behind screening tests (Part 1, Section 1.7), and feature selection (Part 3).

Contents

1	Classification in the KDD Process	4
1.1	What Classification Is	4
1.2	Same Workflow, Different Target	4
2	Foundations of Classification	5
2.1	The Classification Problem	5
2.2	Decision Boundaries	5
2.3	Scores, Probabilities, and Thresholds	6
3	Logistic Regression	6
3.1	From Linear Regression to Probabilities	6
3.2	Interpreting Coefficients: Odds Ratios	6
3.3	Regularization and Multiple Classes	7
4	k-Nearest Neighbors	8
5	Decision Trees	8
5.1	Trees as Rules	8
5.2	How a Tree Chooses Its Splits	8
5.3	Controlling Tree Complexity	9
6	Comparing Classifiers and Model Complexity	9
7	Developing a Classification Model	10

8	Evaluation of Classification Models	11
8.1	The Confusion Matrix	11
8.2	Accuracy, Precision, Recall, Specificity, and F1	11
8.3	Thresholds and the Precision–Recall Trade-off	12
8.4	Threshold-Free Evaluation: ROC and Precision–Recall Curves	12
8.5	More Than Two Classes	13
8.6	Calibration	13
8.7	Choosing and Reporting Metrics	14
9	Python Implementation	14
10	Integrated KDD Case Study: Diabetes Screening	16
11	Common Pitfalls	17
12	Key Takeaways	18

Notation and Terminology Quick Reference

The table below collects the recurring terms of this reading for quick lookup. Each term is developed fully in the section listed. Terms shared with Reading 5.1 — training/validation/test sets, cross-validation, baseline, overfitting, regularization, data leakage — appear in that reading’s quick-reference table.

Term / symbol	Short definition	§
Class label, positive class	The categorical outcome; in binary problems, the class of interest coded 1	1.1
Decision boundary	The surface in feature space where the predicted class changes	2.2
Score $\hat{p}$, threshold t	A predicted probability of the positive class; predict 1 when $\hat{p} \geq t$	2.3
Odds, log-odds (logit)	$p/(1 - p)$ and its logarithm; logistic regression is linear in the log-odds	3.1
Odds ratio	Multiplicative change in odds per unit change of a predictor	3.2
k -nearest neighbors (kNN)	Classify by majority vote of the k closest training points	4
Gini impurity	How mixed the classes are in a node; trees choose splits that reduce it	5.2
Stratified split	A split that preserves the class proportions in every part	7
TP, FP, TN, FN	True/false positives and negatives: the four cells of the confusion matrix	8.1
Precision, recall, specificity	Correct among predicted positives; found among actual positives; correct among actual negatives	8.2
F1 score	Harmonic mean of precision and recall	8.2
ROC curve, AUC	Recall versus false positive rate across all thresholds; area under it	8.4
PR curve, average precision	Precision versus recall across thresholds; its summary area	8.4
Calibration	Whether predicted probabilities match observed frequencies	8.6

Learning Objectives

This reading assumes the EDA readings and Reading 5.1 on regression. After completing it, you should be able to:

- Describe classification as a predictive KDD task, distinguish binary, multiclass, and multilabel problems, and explain the difference between a predicted label and a predicted score.
- Explain how logistic regression, k -nearest neighbors, and decision trees form their decision boundaries, and interpret a logistic coefficient as an odds ratio and a tree as a set of rules.
- Relate each algorithm’s complexity control (regularization, k , tree depth) to underfitting and overfitting, and choose an appropriate algorithm for a problem.
- Build a leakage-free classification workflow with a stratified split, a majority-class baseline, cross-validated tuning, and a single final test evaluation.
- Compute and interpret the confusion matrix, accuracy, precision, recall, specificity, F1, ROC AUC, and average precision, and explain why accuracy can mislead when classes are imbalanced.
- Choose a decision threshold from the costs of different errors, using validation data rather than the test set.

1 Classification in the KDD Process

1.1 What Classification Is

Reading 5.1 introduced regression as the predictive task for a continuous target. **Classification** is the predictive task for a *categorical* target: the model learns, from labeled examples, a rule that assigns each new record to one of a fixed set of **classes** [6]. Within the KDD process [5], classification is probably the most widely deployed data-mining task. Is this tumor malignant? Will this patient be readmitted within 30 days? Is this transaction fraudulent? Which cell type does this single-cell expression profile belong to? Is this field pixel crop or weed?

Classification problems come in three shapes:

- **Binary**: two classes, e.g., diabetic / not diabetic. By convention the class of interest — usually the rarer or more consequential one — is the **positive class**, coded 1; the other is the negative class, coded 0. “Positive” means “the thing we are looking for,” not “good.”
- **Multiclass**: one label out of three or more, e.g., breast-cancer molecular subtype, or land-cover type in a satellite image.
- **Multilabel**: several labels can apply at once, e.g., the set of diagnoses on a discharge summary.

This reading concentrates on binary classification. Its ideas extend to multiclass problems with modest changes (Sections 3.3 and 8.5); multilabel problems are beyond this introduction.

1.2 Same Workflow, Different Target

Everything Reading 5.1 established about the modeling workflow carries over unchanged: define the problem, prepare data inside a pipeline, split before modeling, start from a baseline, tune with cross-validation on training data, test once, interpret honestly. What changes are three things, and they organize this reading:

1. **The output.** A classifier usually produces a *score* (often a probability), and a *threshold* turns the score into a label (Section 2.3).
2. **The algorithms.** Three introductory classifiers with very different logic — logistic regression, k -nearest neighbors, and decision trees (Sections 3–6).
3. **The evaluation.** Errors are no longer distances but mistakes of two kinds, false alarms and misses. Their relative cost determines which metric and which threshold are appropriate (Section 8).

2 Foundations of Classification

2.1 The Classification Problem

The data are n labeled observations (x_i, y_i) , where x_i is a vector of p features and $y_i \in \{0, 1\}$ is the class. The goal is a function $\hat{f}$ that predicts the class of a new observation from its features. As in regression, the labels contain noise: two patients with identical measurements may have different outcomes. So even the best possible classifier makes some errors, and the realistic goal is to approach that floor on data the model has never seen.

2.2 Decision Boundaries

A classifier divides feature space into regions, one per predicted class. The surface separating the regions is the **decision boundary**. Its shape is the clearest way to see how different algorithms “think.” Figure 1 fits four classifiers to the same two features of the Breast Cancer Wisconsin data used in Section 9: the mean radius and mean texture of cell nuclei, for 455 training tumors. Logistic regression draws a straight line. A 1-nearest-neighbor classifier draws a jagged boundary that wraps around individual points. A 15-nearest-neighbor classifier draws a smoother curve. A decision tree draws axis-parallel rectangles.

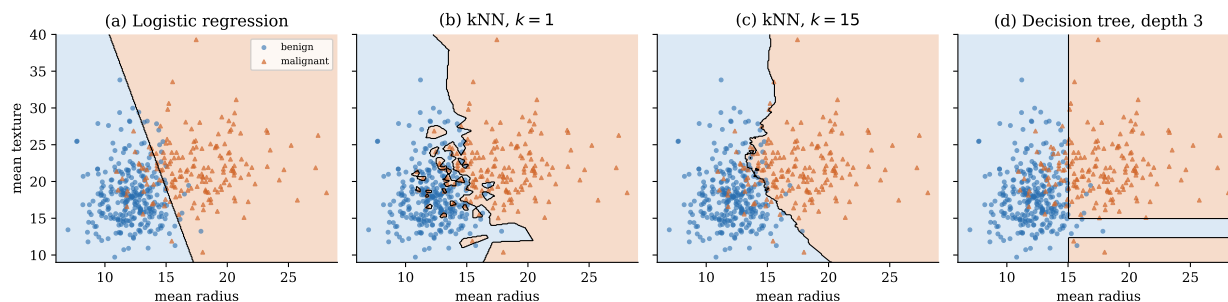


Figure 1: Decision boundaries of four classifiers on two features of the Breast Cancer Wisconsin training data (actual results). Shaded regions show the predicted class. Cross-validated accuracies: (a) 0.89, (b) 0.86 despite 100% training accuracy, (c) 0.90, (d) 0.87.

The figure previews the bias–variance trade-off of Reading 5.1, now in classification form. The 1-NN boundary classifies every training point correctly (training accuracy 1.00), yet its cross-validated accuracy, 0.86, is the lowest of the four: it has memorized noise. The straight line and the 15-NN curve are less flexible and generalize better (0.89 and 0.90).

2.3 Scores, Probabilities, and Thresholds

Most classifiers do not output a label directly. They output a **score** $\hat{p}$, typically an estimate of $P(y = 1 | x)$, the probability that the observation belongs to the positive class. A label then comes from a **threshold** t :

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{p} \geq t, \\ 0 & \text{otherwise.} \end{cases}$$

Software defaults to $t = 0.5$, but that default is a convenience, not a law. A lower threshold flags more cases as positive, catching more true positives at the cost of more false alarms; a higher threshold does the reverse. Separating the *score* (what the model learned) from the *threshold* (a decision about costs) is one of the most useful ideas in applied classification. Section 8.3 returns to it, and the case study in Section 10 chooses a threshold deliberately.

Key Idea

A classifier has two parts: a **scoring model** that ranks observations by how likely they are to be positive, and a **decision rule** (a threshold) that converts scores into actions. Some metrics evaluate the ranking regardless of threshold (ROC AUC, average precision). Others evaluate one particular threshold (accuracy, precision, recall, F1). Report both kinds.

3 Logistic Regression

3.1 From Linear Regression to Probabilities

Why not simply fit the linear regression of Reading 5.1 to a 0/1 target? Because a straight line produces values below 0 and above 1, which cannot be probabilities. **Logistic regression** [3] keeps the familiar linear combination of predictors but passes it through the S-shaped **logistic (sigmoid) function**, which squeezes any real number into $(0, 1)$:

$$\hat{p} = P(y = 1 | x) = \frac{1}{1 + e^{-z}}, \quad z = b_0 + b_1x_1 + \cdots + b_px_p.$$

Equivalently, the model is linear in the **log-odds**, or **logit**:

$$\log \frac{\hat{p}}{1 - \hat{p}} = b_0 + b_1x_1 + \cdots + b_px_p.$$

The **odds** $p/(1 - p)$ compare the chance of the event with the chance of no event: a probability of 0.75 corresponds to odds of 3 (“3 to 1”). The coefficients are not fitted by least squares but by **maximum likelihood**: the values that make the observed labels most probable. Equivalently, they minimize the *log loss*, $-\frac{1}{n} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$, which heavily penalizes confident wrong predictions [12, 10]. Because $\hat{p} = 0.5$ exactly when $z = 0$, the 0.5-threshold decision boundary is the set of points where $b_0 + b_1x_1 + \cdots + b_px_p = 0$: a straight line, plane, or hyperplane, as in Figure 1(a).

3.2 Interpreting Coefficients: Odds Ratios

A worked example (actual results). Fitting logistic regression with a single predictor, plasma glucose in mg/dL, to the 576 training patients of the Pima diabetes case study (Section 10) gives

$$\log \frac{\hat{p}}{1 - \hat{p}} = -5.969 + 0.0427 \times \text{glucose}.$$

For a patient with glucose 150, $z = -5.969 + 0.0427 \times 150 = 0.436$ and $\hat{p} = 1/(1 + e^{-0.436}) = 0.61$. At glucose 100 the predicted probability is 0.16; at 125 it is 0.35; at 175 it is 0.82. The probability crosses 0.5 at $5.969/0.0427 \approx 140$ mg/dL, which is where a default-threshold classifier switches its prediction. Figure 2 shows the curve.

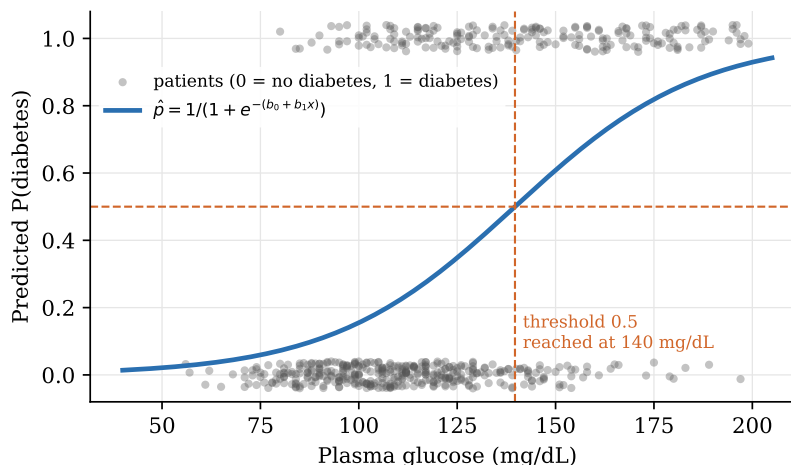


Figure 2: Single-predictor logistic regression of diabetes on plasma glucose, Pima training data (actual results). Points are patients (jittered vertically); the curve is the fitted probability.

A coefficient b_j is the change in *log-odds* per unit of x_j , with the other predictors held constant. That scale is unintuitive, so we exponentiate: e^{b_j} is the **odds ratio**, the factor by which the odds are *multiplied* per unit increase in x_j . Here $e^{0.0427 \times 10} = 1.53$: each additional 10 mg/dL of glucose multiplies the odds of diabetes by about 1.5. Note that the *probability* does not change by a fixed amount, because the curve is steepest near $\hat{p} = 0.5$ and flat at the extremes. Odds ratios above 1 indicate higher risk, below 1 lower risk, and exactly 1 no association. With standardized predictors, e^{b_j} is the odds ratio per standard deviation, which allows rough comparison across features. All the cautions of Reading 5.1 apply: coefficients are partial associations, not causal effects, and collinear predictors make them unstable.

3.3 Regularization and Multiple Classes

Logistic regression can overfit when there are many predictors, and it is regularized exactly as in Reading 5.1 with L_2 (Ridge-style) or L_1 (Lasso-style) penalties. In scikit-learn, `LogisticRegression` applies an L_2 penalty *by default*, with strength controlled by `C`, the *inverse* of the penalty ($C = 1$ by default; smaller C means stronger shrinkage) [17]. Standardize the features first, for the same reasons as before. For more than two classes, the **multinomial** (softmax) extension estimates one linear score per class and converts the scores into probabilities that sum to 1.

Tip

Logistic regression is the right first model for almost any tabular classification problem. It is fast, hard to overfit when regularized, produces reasonably calibrated probabilities, and yields odds ratios that clinicians and domain experts already understand. A more complex model should earn its place by clearly beating it on validation data.

4 k -Nearest Neighbors

The **k -nearest-neighbor (kNN)** classifier [2] is the most direct application of the distance measures of Part 2, Section 3: *similar records tend to have similar labels*. To classify a new observation, find the k training observations closest to it (usually by Euclidean distance on standardized features), and predict the majority class among them. The fraction of neighbors in the positive class serves as the score $\hat{p}$.

A small example (illustrative). A new tumor’s five nearest training tumors are labeled malignant, malignant, benign, malignant, benign. With $k = 5$, three of five neighbors are malignant: $\hat{p} = 0.6$, and the prediction at the default threshold is malignant. With $k = 3$, only the three closest count. If those are malignant, malignant, benign, then $\hat{p} = 0.67$ and the prediction is still malignant, but a different neighbor ordering could flip it. The choice of k matters.

kNN has no training step in the usual sense. It stores the training data and does all its work at prediction time, so it is called a *lazy* learner. This makes it flexible — its boundary can take any shape (Figure 1b,c) — but it also brings three practical requirements:

- **Scale the features.** Distance is dominated by features with large numeric ranges. Unscaled, a feature measured in thousands swamps one measured in fractions. Standardization inside the pipeline is essential, not optional.
- **Choose k by cross-validation.** Small k gives a flexible, high-variance boundary (1-NN memorizes the training set); large k gives a smooth, high-bias one that eventually predicts the majority class everywhere. Odd values of k avoid ties in binary problems.
- **Mind the dimensionality.** As Part 2, Section 3.4 showed, distances lose contrast in high dimensions, so kNN degrades when many features are irrelevant. Feature selection (Part 3) often helps kNN more than any other classifier.

Prediction requires computing distances to every training point, which is slow for very large datasets. Specialized index structures help, but kNN is mostly used on small- to medium-sized tabular data.

5 Decision Trees

5.1 Trees as Rules

A **decision tree** classifies by asking a sequence of yes/no questions about single features [1]. Each internal node tests one feature against a cut-off; each leaf assigns a class, and the class proportions among the training observations in the leaf give the score. Figure 3 shows an actual depth-2 tree learned from the 455 Breast Cancer Wisconsin training tumors (all 30 features available). It reads as a set of rules: *if the worst perimeter exceeds 112.8 and the mean texture exceeds 14.95, predict malignant* (146 of the 151 training tumors in that leaf were malignant). This tree reaches 95.4% training accuracy with two questions per tumor. Rules a clinician can read and challenge are the main reason trees remain popular.

5.2 How a Tree Chooses Its Splits

Trees are grown greedily from the top. At each node the algorithm considers every feature and every possible cut-off, and picks the split that makes the two child nodes as “pure” (single-class) as

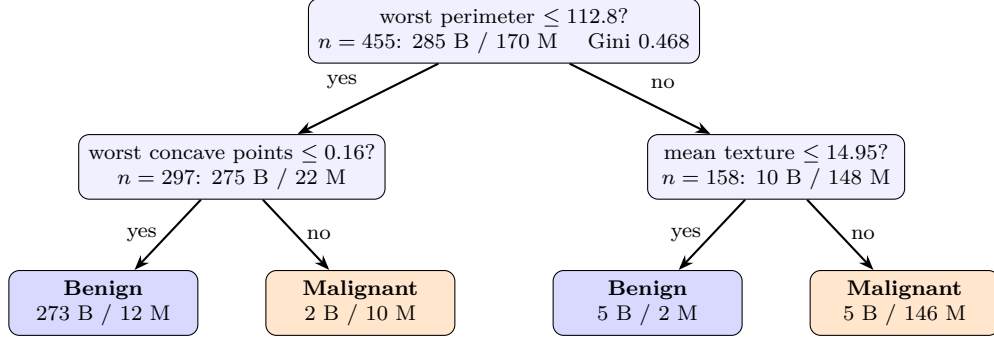


Figure 3: A depth-2 decision tree fitted to the Breast Cancer Wisconsin training data (actual results; B = benign, M = malignant). Each leaf predicts its majority class.

possible. The most common purity measure, used by the CART algorithm and by scikit-learn by default, is **Gini impurity**,

$$\text{Gini} = 1 - \sum_c \pi_c^2,$$

where π_c is the proportion of class c in the node. It is 0 for a pure node and 0.5 for a perfectly mixed binary node.

Worked example (actual counts from Figure 3). The root holds 285 benign and 170 malignant tumors: $\text{Gini} = 1 - (285/455)^2 - (170/455)^2 = 0.468$. The chosen split sends 297 tumors left (275 B, 22 M; $\text{Gini} = 0.137$) and 158 right (10 B, 148 M; $\text{Gini} = 0.119$). The size-weighted impurity of the children is $(297 \times 0.137 + 158 \times 0.119)/455 = 0.131$, a decrease of 0.337. No other single feature and cut-off reduced impurity more, so this split was chosen. The process then repeats inside each child. An alternative criterion, *entropy* (information gain), behaves very similarly in practice [6].

5.3 Controlling Tree Complexity

Left unchecked, a tree keeps splitting until every leaf is pure, which memorizes the training data. Complexity is controlled with hyperparameters tuned by cross-validation: maximum depth (`max_depth`), minimum samples per leaf (`min_samples_leaf`), or cost-complexity pruning (`ccp_alpha`). Trees have attractive properties: they need no feature scaling, handle mixed feature types, capture interactions and nonlinearity naturally, and are readable when shallow. Their main weakness is **instability**. A small change in the training data can change an early split and therefore the whole tree, and single trees are usually less accurate than the other methods. Combining many trees — random forests and gradient boosting — addresses exactly this weakness and is the subject of a later reading.

6 Comparing Classifiers and Model Complexity

Each algorithm has its own complexity control, and each shows the training/validation gap of Reading 5.1. Figure 4 shows it on the full 30-feature Breast Cancer Wisconsin training set. For kNN, training accuracy is perfect at $k = 1$ but cross-validated accuracy peaks at $k = 3$ (0.969) and declines steadily as k grows (0.916 at $k = 101$). For trees, training accuracy climbs to 1.00 by depth 8, while cross-validated accuracy stays between 0.91 and 0.93 at every depth. Extra depth buys training fit, not generalization.

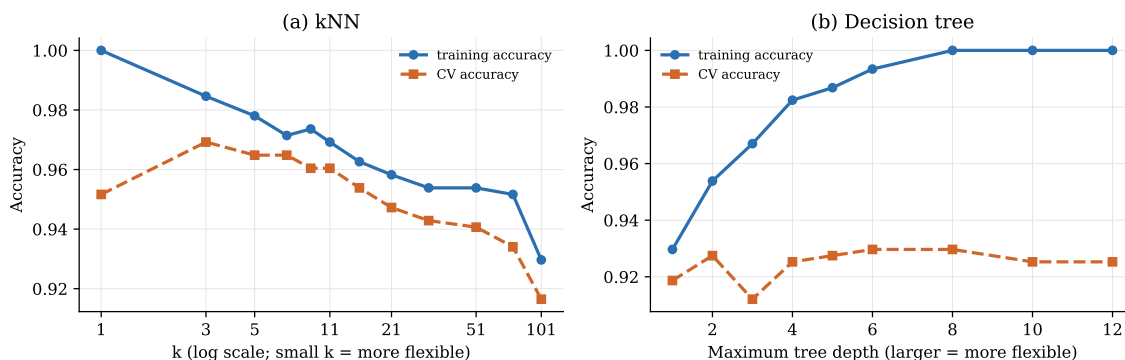


Figure 4: Training versus 5-fold cross-validated accuracy as complexity varies, Breast Cancer Wisconsin training data (actual results).

Table 1 summarizes the three algorithms. A few other families are worth knowing by name. *Naive Bayes* applies Bayes’ theorem with a strong independence assumption and is fast and surprisingly effective for text. *Support vector machines* find a maximum-margin boundary. *Ensembles* (random forests, gradient boosting) combine many trees and are often the most accurate choice for tabular data. *Neural networks* dominate for images, sequences, and text. Later readings cover several of these. The evaluation principles below apply to all of them.

	Logistic regression	k -nearest neighbors	Decision tree
Boundary shape	Linear (in features used)	Any shape, local	Axis-parallel rectangles
Complexity control	Penalty strength C	Number of neighbors k	Depth, leaf size, pruning
Feature scaling	Needed (with penalty)	Essential	Not needed
Interpretability	Odds ratios	Similar cases (“neighbors”)	Readable rules (if shallow)
Probability quality	Usually good	Coarse (fractions of k)	Coarse (leaf proportions)
Main weakness	Misses nonlinearity unless engineered	Slow predictions; high dimensions	Unstable; modest accuracy

Table 1: Three introductory classifiers at a glance.

7 Developing a Classification Model

The eight-step workflow of Reading 5.1, Section 7 applies directly. Classification adds a few specific practices:

Stratify the splits. With 35% positives, a random split could easily give a test set with 28% or 42%, and with rare classes some folds might contain almost no positives. A **stratified** split (`stratify=y` in `train_test_split`, `StratifiedKFold` for cross-validation) preserves the class proportions in every part and gives more stable estimates [14]. Grouped or time-ordered data still need group or time splits, as in regression.

Use the majority-class baseline. The classification counterpart of “predict the mean” is “always predict the most common class” (`DummyClassifier`). Its accuracy equals the majority-class share — 63% for the breast-cancer data, 65% for Pima — so a model with “78% accuracy” is a much smaller achievement than it sounds. The baseline has zero recall and an ROC AUC of exactly 0.5.

Plan for class imbalance. When positives are rare (fraud, rare diseases, equipment failures), first change the *evaluation*: use recall, precision, PR curves, and a cost-based threshold rather than accuracy. Then, if needed, change the *training*, for instance with class weights (`class_weight="balanced"`) or resampling [9]. Any resampling must happen inside the training folds only. Oversampling before splitting places copies of the same patient in both training and validation data, a form of leakage.

Choose the threshold on validation data. The threshold is a tuning decision like any hyperparameter. Choose it from cross-validated (out-of-fold) predictions on the training set to meet a target — for instance “recall at least 0.80” — and then apply that fixed threshold to the test set. Choosing it by looking at test results leaks information, exactly like choosing a model that way [22].

Common Pitfall

Leakage is easier to miss in classification. Besides the preprocessing leakage of Reading 5.1 [13], watch for features that *encode the label*: a medication prescribed only after diagnosis, a billing code entered once fraud was confirmed, a “days in ICU” field for a mortality model. These produce spectacular validation scores and useless deployed models. Ask of every feature: *would its value be known at the moment the prediction is needed?*

8 Evaluation of Classification Models

8.1 The Confusion Matrix

Every thresholded binary prediction falls into one of four cells:

	Predicted positive ($\hat{y} = 1$)	Predicted negative ($\hat{y} = 0$)
Actual positive ($y = 1$)	True positive (TP)	False negative (FN) — a <i>miss</i>
Actual negative ($y = 0$)	False positive (FP) — a <i>false alarm</i>	True negative (TN)

All threshold-based metrics are ratios of these counts. scikit-learn’s `confusion_matrix` prints them as `[[TN, FP], [FN, TP]]`, with the negative class first. Be careful when reading it.

8.2 Accuracy, Precision, Recall, Specificity, and F1

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Accuracy is the fraction of all predictions that are correct. **Precision** (positive predictive value) answers “when the model raises an alarm, how often is it right?” **Recall** (sensitivity, true positive rate) answers “of all actual positives, how many did it find?” **Specificity** (true negative rate) is recall for the negative class. **F1** is the harmonic mean of precision and recall. It is high only when

both are high, which makes it a useful single number when positives are rare and true negatives are uninteresting.

Worked example: the accuracy paradox (illustrative). A screening model is applied to 1,000 people, 50 of whom (5%) have the disease. It finds 40 of them ($TP = 40$, $FN = 10$) and raises 95 false alarms among the 950 healthy people ($FP = 95$, $TN = 855$). Then:

$$\begin{aligned} \text{Accuracy} &= 895/1000 = 0.895, & \text{Precision} &= 40/135 = 0.30, & \text{Recall} &= 40/50 = 0.80, & \text{Specificity} \\ & & & & & & = 855/950 = 0.90, & F_1 &= 0.43. \end{aligned}$$

A useless model that declares everyone healthy has accuracy $950/1000 = 0.95$ — *higher* than the real model — with recall 0. Accuracy rewards agreeing with the majority, so when classes are imbalanced it says little about whether the model does its job. The low precision is the base-rate effect of Part 1, Section 1.7: when a condition is rare, even a fairly specific test produces more false alarms than true detections. For screening, that may be acceptable if a confirmatory test follows. This is a judgment about costs, not a statistical one.

8.3 Thresholds and the Precision–Recall Trade-off

Every metric in Section 8.2 depends on the threshold. Lowering it moves cases from predicted-negative to predicted-positive. Recall can only rise, specificity can only fall, and precision usually falls. Figure 5 shows this for the logistic model of the case study. No threshold is “correct” in general. The right one depends on what a miss costs relative to a false alarm. A missed cancer is far more serious than an unnecessary biopsy; a spam filter that hides a legitimate email is worse than one that lets an advertisement through.

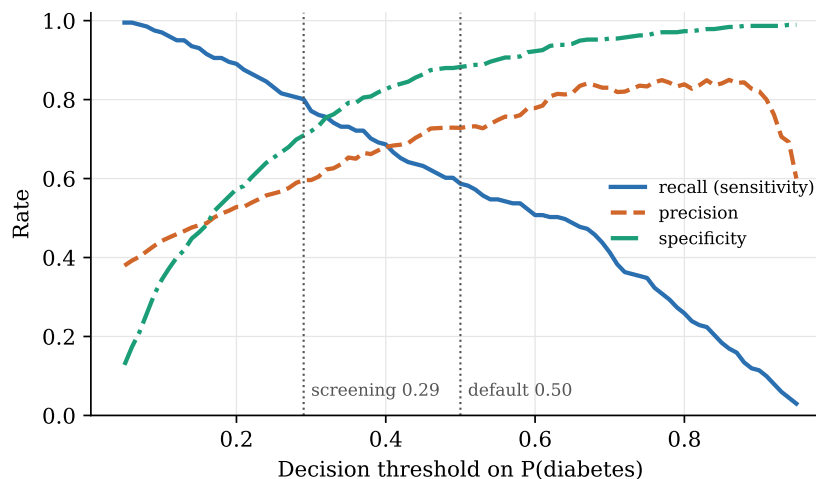


Figure 5: Recall, precision, and specificity as functions of the threshold, from cross-validated (out-of-fold) predictions of the Pima logistic model on training data (actual results). The 0.29 threshold is the largest that keeps recall at or above 0.80.

8.4 Threshold-Free Evaluation: ROC and Precision–Recall Curves

To evaluate the *scoring model* independently of any one threshold, sweep the threshold from 1 down to 0 and trace the results [4].

The **ROC curve** plots recall (true positive rate) against the false positive rate ($1 - \text{specificity}$). A model with no skill follows the diagonal; a perfect model goes straight up to the top-left corner. The **area under the ROC curve (AUC)** summarizes the curve in one number and has a useful interpretation: it is the probability that a randomly chosen positive receives a higher score than a randomly chosen negative. AUC 0.5 means random ranking, 1.0 perfect ranking. Because both its axes are rates within a single class, ROC AUC does not change with class prevalence. This is an advantage for comparing models across datasets, and a drawback when positives are rare, because a large number of false alarms can still look like a small false positive *rate*.

The **precision–recall (PR) curve** plots precision against recall. Its no-skill reference is a horizontal line at the prevalence, not at 0.5, and its summary is **average precision (AP)**. Because precision is computed among predicted positives, the PR curve exposes exactly the false-alarm burden that ROC can hide. Saito and Rehmsmeier showed that it is the more informative plot for imbalanced problems [18]. Figure 6 shows both curves for the case-study models.

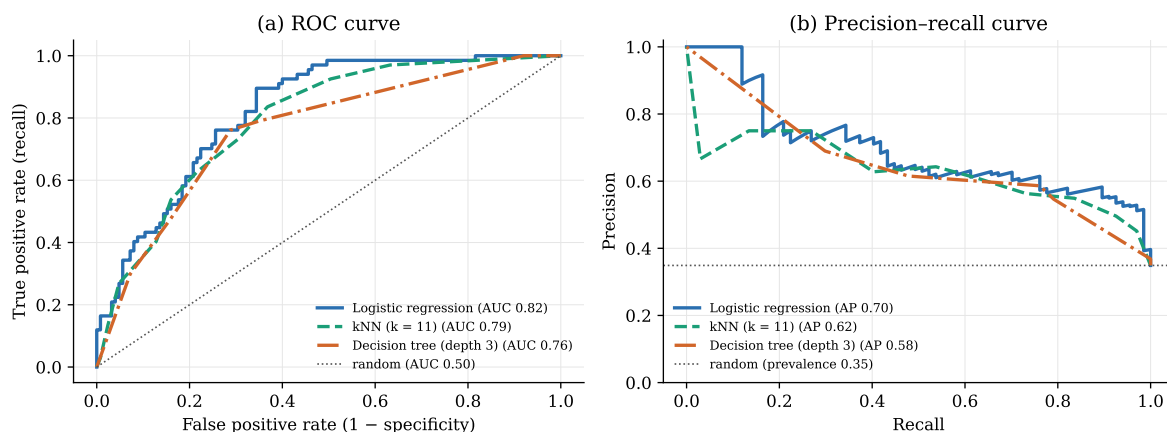


Figure 6: ROC and precision–recall curves on the Pima test set for the three case-study classifiers (actual results). The dotted lines are the no-skill references: the diagonal for ROC, and the 0.35 prevalence for PR.

8.5 More Than Two Classes

For multiclass problems the confusion matrix becomes $K \times K$, and precision, recall, and F1 are computed per class by treating each class in turn as “positive versus the rest.” The per-class values are combined by **macro averaging** (a simple mean, so every class counts equally, which highlights poor performance on small classes) or **weighted averaging** (weighted by class size, which is dominated by large classes). `classification_report` prints both. Report the per-class values whenever a minority class matters.

8.6 Calibration

For risk communication, the probabilities themselves must be trustworthy: among patients given $\hat{p} \approx 0.3$, about 30% should turn out positive. A model with this property is **calibrated**. Ranking metrics like AUC ignore calibration entirely, so a model can rank well and still systematically over- or under-state risk. Logistic regression tends to be reasonably calibrated; kNN and trees produce coarse probabilities, and some other learners are systematically miscalibrated [16]. In health

applications calibration deserves explicit checking, for example with a reliability plot of observed frequency against predicted probability [21].

8.7 Choosing and Reporting Metrics

Metric	Answers	Threshold?	Use when / caveats
Accuracy	What fraction of predictions is correct?	Yes	Balanced classes, equal error costs; misleading otherwise
Precision	How trustworthy is an alarm?	Yes	False alarms are costly (alerts, interventions)
Recall	How many positives are found?	Yes	Misses are costly (screening, safety)
Specificity	How many negatives are cleared?	Yes	Paired with recall in clinical reporting
F1	Balance of precision and recall	Yes	Rare positives; ignores true negatives
ROC AUC	How well are cases ranked?	No	Comparing scoring models; optimistic with rare positives
Average precision	Ranking quality on the positives	No	Imbalanced data; compare to prevalence

Table 2: Classification metrics at a glance.

A complete report gives the confusion matrix at the chosen threshold, recall and precision (plus specificity in clinical settings), one threshold-free metric (ROC AUC, and AP when positives are rare), the same numbers for the majority-class baseline, and the cross-validation spread.

9 Python Implementation

The example applies the full workflow to the **Breast Cancer Wisconsin (Diagnostic)** dataset [20], distributed with scikit-learn [17]. Each of its 569 rows describes a breast mass from a digitized image of a fine-needle aspirate. Ten nuclear characteristics (radius, texture, perimeter, concavity, and so on) are each summarized as a mean, a standard error, and a “worst” value, giving 30 features. 212 masses (37%) are malignant. The code uses pandas [15], NumPy [7], Matplotlib [11], and scikit-learn.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     GridSearchCV, cross_validate)

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, RocCurveDisplay)
```

```

# 1-3. Load data; make "malignant" the positive class (1)
data = load_breast_cancer(as_frame=True)
X = data.data
y = (data.target == 0).astype(int)      # sklearn codes malignant as 0
print(X.shape, "malignant share:", round(y.mean(), 3))  # (569, 30) 0.373

# 4. Stratified split keeps the 37% malignant share in both parts
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 5. Baseline + three classifiers; scaling lives inside each pipeline
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
models = {
    "Baseline": DummyClassifier(strategy="most_frequent"),
    "Logistic": make_pipeline(StandardScaler(),
                              LogisticRegression(max_iter=1000)),
    "kNN": GridSearchCV(
        make_pipeline(StandardScaler(), KNeighborsClassifier()),
        {"kneighborsclassifier__n_neighbors": [1, 3, 5, 7, 9, 11, 15, 21, 31]},
        cv=cv, scoring="roc_auc"),
    "Tree": GridSearchCV(
        DecisionTreeClassifier(random_state=0),
        {"max_depth": [1, 2, 3, 4, 5, 6, 8, None]},
        cv=cv, scoring="roc_auc"),
}

# 6. Cross-validation on the training set (the GridSearchCV inside each
#    fold makes this a nested CV: tuning never sees the scoring fold)
metrics = ["accuracy", "precision", "recall", "f1", "roc_auc"]
rows = {}
for name, model in models.items():
    s = cross_validate(model, X_train, y_train, cv=cv, scoring=metrics)
    rows[name] = {m: s[f"test_{m}"].mean() for m in metrics}
print(pd.DataFrame(rows).T.round(3))

# 7. Refit the chosen model on all training data; evaluate ONCE on test
best = models["Logistic"].fit(X_train, y_train)
proba = best.predict_proba(X_test)[: , 1]      # P(malignant)
pred = (proba >= 0.5).astype(int)
print(confusion_matrix(y_test, pred))          # [[TN FP] [FN TP]]
print(classification_report(y_test, pred,
                             target_names=["benign", "malignant"]))
print("test ROC AUC:", round(roc_auc_score(y_test, proba), 3))

# 8. A lower threshold trades false positives for fewer missed cancers
for t in [0.5, 0.3]:
    print(t, confusion_matrix(y_test, (proba >= t).astype(int)).ravel())

# 9. ROC curve and the largest standardized coefficients (log-odds)
RocCurveDisplay.from_predictions(y_test, proba)
coef = pd.Series(best[-1].coef_[0], index=X.columns).sort_values()
print(coef.tail(5).round(2))

```

What the code shows (actual output, scikit-learn 1.9). Table 3 gives the nested cross-validation results on the 455 training tumors.

Model	Accuracy	Precision	Recall	F1	ROC AUC
Baseline (majority class)	0.626	0.000	0.000	0.000	0.500
Logistic regression	0.978	0.983	0.959	0.970	0.995
kNN (tuned k)	0.945	0.981	0.871	0.922	0.986
Decision tree (tuned depth)	0.930	0.921	0.888	0.904	0.926

Table 3: Stratified 5-fold nested cross-validation on the Breast Cancer Wisconsin training set (actual results; malignant = positive).

Logistic regression is best on every metric, so it is selected. It is also the simplest model to explain. Two details deserve attention. First, the three classifiers differ more in *recall* (0.87 to 0.96) than in accuracy, and recall is what matters most for a cancer diagnosis. Second, the tree’s ROC AUC (0.93) lags far behind the other two (0.99) even though its accuracy is only a few points lower, because a shallow tree yields only a handful of distinct scores, which makes its ranking coarse. (Tuning by AUC selected $k = 21$ for kNN and depth 2 for the tree when refitted on the full training set.)

On the untouched test set of 114 tumors, the logistic model’s confusion matrix at threshold 0.5 is TN = 71, FP = 1, FN = 3, TP = 39: accuracy 0.965, precision 0.975, recall 0.929, F1 0.951, and ROC AUC 0.996. The three missed malignancies are the errors a clinician would worry about. Lowering the threshold to 0.3 reduces the misses to 1 without adding any false positives on this test set. *That comparison is shown for illustration only.* In a real study the threshold would be fixed beforehand from cross-validated predictions, as the case study below does, because choosing it by inspecting test results would contaminate the test estimate. The largest standardized coefficients belong to worst texture, radius error, worst symmetry, and mean concave points. With 30 strongly correlated measurements of the same nuclei, however, individual coefficients should not be over-interpreted, for the collinearity reasons of Reading 5.1, Section 4.2.

10 Integrated KDD Case Study: Diabetes Screening

Real-World Case Study

Problem. A community health program wants to flag adults at high risk of diabetes from inexpensive routine measurements, so that they can be referred for confirmatory testing. A missed case delays treatment; a false alarm costs one extra test. The program therefore sets a goal of catching *at least 80% of true cases*, accepting more false alarms.

Data and target. The Pima Indians Diabetes data, from the study by Smith and colleagues [19], describe 768 women aged 21 or older of Pima Indian heritage. The binary target records diabetes by World Health Organization criteria; 268 women (34.9%) are positive. There are eight features: number of pregnancies, plasma glucose, diastolic blood pressure, triceps skinfold thickness, serum insulin, BMI, a diabetes pedigree (family-history) score, and age. All results below are *actual results* from a stratified 75/25 split (`random_state=42`; 576 training and 192 test women) with stratified 5-fold cross-validation on the training set; tuning used nested grid search.

EDA and preparation. EDA finds physiologically impossible zeros: glucose (5 records), blood pressure (35), BMI (11), skinfold (227), and insulin (374, nearly half the data). These

are missing values recorded as 0, not measurements. Here we recode them as missing and impute training-fold medians inside the pipeline. This improved the cross-validated ROC AUC of logistic regression only modestly (0.840 versus 0.829 with the zeros left in), but it removes nonsense values from the features and their interpretation. Glucose ($r = 0.47$) and BMI (0.29) correlate most with the outcome. Data from a single community also limit how far any model can be generalized to other populations, a point the program must document.

Models and evaluation. Cross-validated results on the training set:

	Accuracy	Precision	Recall	F1	ROC AUC	AP
Baseline (majority)	0.651	0.000	0.000	0.000	0.500	0.349
Logistic regression	0.780	0.730	0.588	0.648	0.840	0.733
kNN ($k = 11$)	0.766	0.694	0.593	0.635	0.827	0.703
Decision tree (depth 3)	0.733	0.638	0.593	0.609	0.765	0.623

This is a much harder problem than the breast-cancer data. The best model improves accuracy over the baseline by only 13 points, and at the default threshold it finds fewer than 60% of the cases — well short of the program’s goal. Logistic regression ranks best (AUC 0.84, fold SD 0.04) and is selected.

Choosing the threshold. Using out-of-fold predicted probabilities on the *training* set (Figure 5), the largest threshold that keeps recall at or above 0.80 is 0.29. That threshold was fixed *before* touching the test set. The test-set confusion matrices compare it with the default:

Threshold	TP	FN	FP	TN	Recall	Precision	Specificity	Accuracy
0.50 (default)	34	33	20	105	0.51	0.63	0.84	0.72
0.29 (screening)	52	15	39	86	0.78	0.57	0.69	0.72

Moving to 0.29 finds 18 more of the 67 diabetic women in the test set at the cost of 19 more referrals, and accuracy is unchanged. Accuracy is blind to precisely the trade-off the program cares about. Test recall (0.78) is close to the 0.80 target, as expected for an estimate carried over from validation. Test ROC AUC is 0.82 and AP 0.70 (against a no-skill AP of 0.35), consistent with cross-validation. Figure 6 shows that logistic regression dominates the tree across nearly all thresholds. The mean predicted probability on the test set (0.345) matches the observed rate (0.349), a first sign of reasonable calibration.

Interpretation. With standardized features, the odds ratios per standard deviation are 3.2 for glucose, 2.1 for BMI, 1.6 for pregnancies, 1.2 for the pedigree score, and 1.1 for age. Blood pressure, skinfold, and insulin are close to 1 once the others are included. Each 30 mg/dL (one SD) higher glucose roughly triples the odds of diabetes with the other factors held fixed — a strong association consistent with clinical knowledge, but an association all the same.

Knowledge and decision. The KDD outcome is a screening rule: compute the logistic risk score from glucose, BMI, pregnancies, family history, and age, and refer anyone above 0.29. Expected performance is about 78% of cases found, with slightly more than half of referrals (precision 0.57) turning out to be diabetic. Before deployment, the program should validate the model on its own population, check calibration within subgroups, and revisit the threshold if the cost of confirmatory testing changes.

11 Common Pitfalls

- **Reporting accuracy alone.** With imbalanced classes, a do-nothing model can score high. Always show the majority-class baseline, recall, and precision.

- **Treating 0.5 as the threshold.** The default ignores error costs. Choose the threshold deliberately, on validation data.
- **Tuning the threshold or model on the test set.** Any choice informed by test results biases the final estimate [22].
- **Unstratified splits.** With rare classes, folds can end up with very few positives and give unstable estimates. Use stratified splitting.
- **Resampling or scaling before splitting.** SMOTE, oversampling, scaling, and imputation belong inside the training folds.
- **Features that encode the label.** Post-outcome information produces excellent but meaningless scores.
- **Forgetting to scale for kNN and penalized logistic regression.** Distances and penalties both depend on units.
- **Confusing the positive class.** scikit-learn’s `load_breast_cancer` codes malignant as 0. Precision and recall refer to whichever class is labeled 1. Check before interpreting.
- **Over-reading ROC AUC with rare positives.** A high AUC can coexist with poor precision. Add the PR curve.
- **Ignoring missing-value codes.** Zeros, `-999`, or “unknown” entries masquerading as measurements, as in the Pima data, distort both models and interpretation.
- **Interpreting coefficients or trees causally or too literally.** Collinearity makes coefficients unstable, and small data changes can reshape a tree.

12 Key Takeaways

1. Classification predicts a categorical label. It shares the regression workflow, but its outputs are scores plus a threshold, and its errors are misses and false alarms.
2. The decision boundary shows how a classifier generalizes: linear for logistic regression, local and flexible for kNN, rectangular for trees.
3. Logistic regression models the log-odds linearly. Its exponentiated coefficients are odds ratios, and it is the right first model for tabular data.
4. kNN classifies by similarity. It needs scaled features and a cross-validated k , and it suffers in high dimensions.
5. Decision trees choose impurity-reducing splits and read as rules. Depth must be limited, and single trees are unstable.
6. Every classifier trades bias for variance through its own complexity control. Training accuracy is not evidence of generalization.
7. Stratify splits, compare against the majority-class baseline, keep resampling and preprocessing inside training folds, and test once.
8. Accuracy misleads when classes are imbalanced. Report the confusion matrix, recall, precision, specificity, and F1 at the chosen threshold.

9. ROC AUC and average precision evaluate ranking independently of the threshold; prefer the PR view when positives are rare.
10. The threshold is a cost decision made on validation data. Calibration matters whenever probabilities are communicated as risks.

Wrap-Up: Connecting Back

Classification reuses nearly everything the course has built. kNN is the distance measures of Part 2, Section 3 turned into a predictor, and its weakness in high dimensions is the curse of dimensionality of Part 2, Section 3.4, which feature selection (Part 3) helps relieve. Logistic regression is the linear model of Reading 5.1 viewed through a sigmoid, with the same regularization, scaling, and collinearity cautions. The accuracy paradox is the base-rate reasoning of Part 1, Section 1.7. And the protocol — stratified split, baseline, nested cross-validation, one test evaluation, decisions made on validation data — is Reading 5.1’s workflow with a categorical target. Later readings add ensembles and other classifiers. They change the model, not the reasoning.

Review Questions

1. For each task, state whether it is binary, multiclass, multilabel, or regression: (a) predicting a patient’s systolic blood pressure; (b) assigning a tumor to one of four molecular subtypes; (c) tagging a news article with all relevant topics; (d) flagging a soil-moisture sensor reading as faulty or valid.
2. A logistic model has $\log(\hat{p}/(1 - \hat{p})) = -4 + 0.8x$. Compute $\hat{p}$ at $x = 5$, the odds ratio for a one-unit increase in x , and the value of x at which a 0.5 threshold switches the prediction.
3. Using Figure 3, classify a tumor with worst perimeter 120, worst concave points 0.12, and mean texture 13.0. What fraction of training tumors in that leaf were malignant, and how confident should you be in this prediction?
4. Compute the Gini impurity of a node with 30 positives and 10 negatives, and of a node with 20 of each. Which is purer?
5. Explain why a 1-nearest-neighbor classifier always has 100% training accuracy (assuming no duplicate points with different labels), and why that number is useless for model selection.
6. A fraud model has 99.2% accuracy on data with 0.5% fraud. What additional numbers would you ask for before deciding whether the model is useful, and why?
7. From this confusion matrix — TP = 45, FN = 15, FP = 30, TN = 910 — compute accuracy, precision, recall, specificity, and F1. Then describe a setting where this model would be acceptable and one where it would not.
8. Two models have ROC AUC 0.91 and 0.89, but the second has higher average precision. Under what data conditions can this happen, and which would you prefer for a rare-disease screen?
9. A colleague tunes the threshold to maximize F1 on the test set and reports the resulting F1. What is wrong, and how should the threshold be chosen?
10. Why is stratification more important for a dataset with 2% positives than for one with 40% positives?

11. In the Pima case study, moving the threshold from 0.50 to 0.29 left accuracy unchanged at 0.72. Explain how that is possible and what it says about accuracy as a selection criterion here.
12. You must explain a model to a clinical committee that needs to see *why* each patient is flagged. Which of the three algorithms in this reading would you propose, and what would you lose compared with the others?

Suggested Hands-on Exercise

Building and defending a classifier (individual or pairs). Choose a binary classification dataset from your project domain, with at least 500 records, at least 6 features, and a positive class below 40%, or use the instructor-provided dataset. Then:

1. **Define the problem:** the positive class, who acts on the prediction, and the relative cost of a miss versus a false alarm.
2. **Perform focused EDA:** class balance, missing-value codes, the features most associated with the label, and any feature that might encode the label.
3. **Split** with stratification and set the test set aside. Build a majority-class baseline.
4. **Train three classifiers** — logistic regression, kNN, and a decision tree — inside pipelines, tuning C , k , and depth with stratified cross-validation.
5. **Compare** the models on recall, precision, F1, ROC AUC, and average precision (cross-validated mean and SD), and plot ROC and PR curves.
6. **Choose a threshold** from out-of-fold predictions on the training set that reflects your stated costs, and justify it.
7. **Evaluate once** on the test set at that threshold: confusion matrix and all metrics.
8. **Interpret** the selected model: odds ratios or tree rules, with collinearity and causation caveats.
9. **Recommend** a deployable rule in a short paragraph for a non-technical stakeholder, including expected misses and false alarms per 1,000 cases and where the model should not be trusted.

Submit a notebook and a two-page summary. Grading emphasizes problem framing, leakage-free design, a cost-justified threshold, and honest interpretation over raw performance.

References

- [1] Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
- [2] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
- [3] Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2), 215–242. <https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>
- [4] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>
- [5] Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54. <https://doi.org/10.1609/aimag.v17i3.1230>

- [6] Han, J., Pei, J., & Tong, H. (2023). *Data Mining: Concepts and Techniques* (4th ed.). Morgan Kaufmann.
- [7] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [8] Hasan, K. (2026). *Knowledge Discovery and Data Mining*. Grand Valley State University. Available at: https://github.com/mdkamrulhasan/data_mining_kdd
- [9] He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- [10] Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied Logistic Regression* (3rd ed.). Wiley. <https://doi.org/10.1002/9781118548387>
- [11] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [12] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- [13] Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), Article 15. <https://doi.org/10.1145/2382577.2382579>
- [14] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, Vol. 2, 1137–1145.
- [15] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- [16] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 625–632. <https://doi.org/10.1145/1102351.1102430>
- [17] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [18] Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- [19] Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 261–265.
- [20] Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. In *Biomedical Image Processing and Biomedical Visualization*, Proc. SPIE 1905, 861–870. <https://doi.org/10.1117/12.148698>
- [21] Van Calster, B., McLernon, D. J., van Smeden, M., Wynants, L., & Steyerberg, E. W. (2019). Calibration: The Achilles heel of predictive analytics. *BMC Medicine*, 17, 230. <https://doi.org/10.1186/s12916-019-1466-7>
- [22] Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, 91. <https://doi.org/10.1186/1471-2105-7-91>